

Relazione - Versione 6: Modello Linguistico Ricorrente

Introduzione

La Versione 6 estende il modello neurale di linguaggio a caratteri in TensorFlow della Versione 5 introducendo una semplice rete neurale ricorrente e uno stato nascosto che trasporta informazioni attraverso un contesto ordinato.

Corpus, vocabolario, finestre di contesto di quattro caratteri, embedding appresi, split di training e validation, mini-batch mescolati, obiettivo cross-entropy, differenziazione automatica e campionamento autoregressivo restano invariati. Invece di appiattire i quattro embedding del contesto in un unico vettore, il modello li elabora un carattere alla volta attraverso uno stato nascosto ricorrente.

```
Caratteri
↓
Identificatori interi
↓
Contesto di 4 caratteri
↓
Embedding addestrabili
↓
Stati nascosti ricorrenti
↓
Pesi di output
↓
Testo generato
```

Obiettivo della Versione 6

Il modello deve essere in grado di:

- riutilizzare lo stesso corpus di 440 caratteri, il vocabolario di 27 caratteri e le finestre di contesto di quattro caratteri della Versione 5;
- riutilizzare embedding addestrabili a 8 dimensioni per i caratteri;
- elaborare sequenzialmente i quattro embedding invece di appiattirli;
- trasportare informazioni attraverso uno stato nascosto ricorrente a 16 dimensioni;
- apprendere embedding, pesi input-to-hidden, hidden-to-hidden e hidden-to-output con `tf.GradientTape`;
- mantenere training mini-batch, monitoraggio della validation e generazione autoregressiva riproducibile;
- verificare stati ricorrenti, forme del modello, gradienti, aggiornamenti dei parametri, retraining deterministico e generazione.

Architettura

Il flusso completo della Versione 6 è:

```
Corpus: 440 caratteri
↓
Vocabolario: 27 caratteri
↓
436 esempi contesto-target da 4 caratteri
↓
Tensore input intero: (436, 4)
↓
Matrice di embedding: (27, 8)
↓
Elaborazione ricorrente sequenziale
↓
Stato nascosto finale: 16 valori
↓
Matrice pesi di output: (16, 27)
↓
Split esempi 80 / 20 riproducibile
↓
Aggiornamenti mini-batch con GradientTape
↓
Generatore autoregressivo a finestra scorrevole
```

Il contesto contiene ancora quattro caratteri precedenti, ma i loro embedding non vengono più concatenati in una rappresentazione appiattita. La Versione 6 elabora il contesto da sinistra a destra e riutilizza gli stessi pesi ricorrenti in ogni posizione.

Finestre di contesto, split e mini-batch

Le finestre scorrevoli creano gli stessi 436 esempi input-target della Versione 5. Ogni esempio contiene quattro identificatori di carattere ordinati. Lo stato nascosto parte da zero per ogni esempio di contesto e viene aggiornato una volta per ciascun carattere.

```
esempi training = 348
esempi validation = 88
dimensione batch = 32
aggiornamenti per epoca = 11
```

Solo gli esempi di training vengono mescolati e usati per aggiornare i parametri. Gli esempi di validation misurano la loss senza modificare il modello. Le finestre di contesto vicine continuano a sovrapporsi, quindi lo split casuale a livello di esempio mantiene la stessa limitazione intenzionale del piccolo dataset della Versione 5.

Embedding addestrabili e pesi ricorrenti

Ogni identificatore di carattere seleziona una riga dalla matrice di embedding addestrabile. I quattro vettori selezionati vengono elaborati sequenzialmente. In ogni posizione, l'embedding corrente e lo stato nascosto precedente vengono combinati e passati attraverso tanh per produrre il nuovo stato nascosto.

```
Forma tensore input: (436, 4)
Forma matrice embedding: (27, 8)
Forma contesto embedded: (1, 4, 8)
Forma matrice pesi input: (8, 16)
Forma matrice pesi ricorrenti: (16, 16)
Forma stato nascosto finale: (1, 16)
Forma matrice pesi output: (16, 27)
Parametri addestrabili: 1032
```

La matrice di embedding contiene 216 valori, la matrice input-to-hidden 128, quella ricorrente 256 e la matrice di output 432. Il modello contiene quindi 1032 parametri addestrabili, leggermente meno dei 1080 parametri usati nella Versione 5.

Loss stabile in TensorFlow

Gli identificatori del contesto vengono mappati negli embedding ed elaborati ricorrentemente. Lo stato nascosto finale viene moltiplicato per i pesi di output per produrre i logits del carattere successivo. La sparse softmax cross-entropy misura l'errore di previsione direttamente dai logits e dagli ID target interi.

```
hidden_state, logits = recurrent_forward(
    embedding_matrix, input_weights,
    recurrent_weights, output_weights, inputs
)

example_losses = tf.nn.sparse_softmax_cross_entropy_with_logits(
    labels=targets, logits=logits
)

loss = tf.reduce_mean(example_losses)
```

Le loss iniziali di training e validation sono entrambe circa 3.2958, coerenti con una previsione quasi uniforme sul vocabolario di 27 caratteri.

Esecuzione controllata e riproducibilità

Il notebook mantiene l'esecuzione su CPU e sopprime i log TensorFlow di basso livello. Seed espliciti controllano l'inizializzazione dei parametri TensorFlow, split dei dati NumPy, shuffle dei mini-batch e campionamento del testo. Le esecuzioni ripetute sono deterministiche nello stesso ambiente software.

Differenziazione automatica

Per ogni mini-batch, `tf.GradientTape` registra i calcoli forward ricorrenti e deriva la loss rispetto a tutte e quattro le matrici di parametri addestrabili:

- `tf.GradientTape` registra embedding lookup, aggiornamenti dello stato nascosto ricorrente, proiezione di output e loss;
- `tape.gradient` calcola i gradienti per embedding, pesi di input, ricorrenti e di output;
- il gradiente sparso degli embedding viene convertito in un tensore denso per questo piccolo vocabolario;
- `assign_sub` applica aggiornamenti di gradient descent a tutte e quattro le variabili;
- le loss di training e validation vengono misurate dopo ogni epoca.

```
with tf.GradientTape() as tape:
    batch_loss = calculate_loss(...)
    embedding_gradient, input_gradient, recurrent_gradient, output_gradient = \
        tape.gradient(batch_loss, [embedding_matrix, input_weights,
                                   recurrent_weights, output_weights])
```

Configurazione training:

```
Learning rate: 1.0
Dimensione batch: 32
Epoche: 100
Seed: 42
```

Risultato del training

```
Loss training iniziale: 3.295837
Loss training finale: 1.525101
Loss validation iniziale: 3.295840
Loss validation finale: 3.035164
Migliore loss validation: 2.839605
Migliore epoca validation: 92
```

Il modello ricorrente apprende lentamente nella prima parte del training. Dopo circa 50 epoche, la loss di training inizia a diminuire chiaramente e continua a migliorare fino all'epoca 100. Anche la loss di validation migliora e raggiunge il minimo all'epoca 92.

Dopo l'epoca migliore, la loss di validation aumenta leggermente mentre quella di training continua a diminuire. Questo mostra l'inizio dell'overfitting, ma è molto meno pronunciato rispetto alla Versione 5 all'epoca 100. Il notebook registra l'epoca migliore per l'analisi ma usa intenzionalmente i parametri finali dell'epoca 100 per la generazione.

Controlli su gradienti e parametri

I test verificano gradienti finiti per tutte e quattro le matrici di parametri addestrabili, forme dei gradienti corrispondenti, stati ricorrenti finiti e che ogni matrice di parametri cambi realmente durante il training.

```
embedding_gradient, input_gradient, recurrent_gradient, output_gradient = \
    tape.gradient(loss, [embedding_matrix, input_weights,
                        recurrent_weights, output_weights])

assert recurrent_gradient.shape == recurrent_weights.shape
assert not np.allclose(recurrent_weights.numpy(),
                       trained_recurrent_weights.numpy())
```

Probabilità apprese

Dopo il training, il modello produce una distribuzione di probabilità del carattere successivo a partire da un contesto ordinato di quattro caratteri. Per il contesto mode, le probabilità più alte sono:

```
spazio: 0.5088    l: 0.1632    i: 0.0957
a: 0.0596        s: 0.0581    t: 0.0311
```

Generazione neurale del testo

Per ogni previsione, la finestra corrente di quattro caratteri viene mappata negli embedding ed elaborata da sinistra a destra. Lo stato nascosto parte da zero per quella finestra, lo stato finale produce i logits e un generatore NumPy locale campiona il carattere successivo. Il carattere più vecchio del contesto viene quindi rimosso e la finestra avanza.

```
mode -> carattere campionato
ode? -> carattere successivo
...
```

Cinque campioni dal contesto addestrato mode con seed 42 sono l, spazio, l, k e spazio. La dimostrazione genera poi 300 caratteri dai parametri finali e lo stesso seed riproduce lo stesso testo.

Controlli inclusi

Le asserzioni del notebook verificano costruzione delle finestre di contesto, tipi di tensori e variabili, forme di tutti i parametri ricorrenti, stati nascosti finiti, split dei dati, loss finite, probabilità normalizzate, gradienti finiti, modifiche dei parametri, retraining deterministico, generazione riproducibile, lunghezza di output richiesta e contesto iniziale.

Un'esecuzione completa dall'inizio alla fine termina con il messaggio All checks passed.

Cosa insegna la Versione 6

La progressione centrale è:

contesto -> ID -> embedding -> stati nascosti ricorrenti -> logits
-> loss -> gradienti -> aggiorna -> scorri -> campiona

La Versione 6 introduce l'elaborazione ricorrente delle sequenze e il primo stato nascosto del progetto. Il focus didattico passa da una rappresentazione fissa appiattita a un calcolo ordinato che trasporta informazioni apprese da una posizione del contesto alla successiva.

Interpretazione

Gli stessi quattro embedding del contesto vengono ora elaborati sequenzialmente. Una matrice ricorrente condivisa viene riutilizzata in ogni posizione, mentre tanh introduce una trasformazione nascosta non lineare. Lo stato nascosto finale a 16 dimensioni riassume il contesto ordinato prima della proiezione di output.

Il modello contiene 1032 parametri, leggermente meno della Versione 5, quindi il cambiamento architetturale non è semplicemente un aumento della dimensione del modello. Anche la dinamica del training cambia sostanzialmente: l'apprendimento inizia lentamente e la migliore loss di validation viene raggiunta molto più tardi, all'epoca 92.

Limitazioni

- il contesto mantiene una lunghezza fissa di quattro caratteri;
- lo stato nascosto parte da zero per ogni esempio di training;
- anche la generazione ricalcola ogni finestra di quattro caratteri da uno stato nascosto zero invece di preservare lo stato indefinitamente;
- il corpus è piccolo ed è incorporato direttamente nel notebook;
- finestre di contesto sovrapposte possono comparire tra training e validation a causa dello split casuale;
- l'unità ricorrente è una semplice trasformazione tanh senza gate né termini di bias;
- non ci sono early stopping o ripristino dei checkpoint, quindi la generazione usa i parametri dell'epoca 100 anche se la validation è migliore all'epoca 92;
- il testo generato cattura pattern sequenziali locali ma non ha ancora coerenza a lungo termine.

Conclusione

La Versione 6 sostituisce la rappresentazione appiattita del contesto della Versione 5 con un'elaborazione ricorrente esplicita. Gli ID dei caratteri selezionano embedding addestrabili, gli embedding aggiornano uno stato nascosto condiviso da sinistra a destra e lo stato finale predice il carattere successivo.

Il semplice meccanismo ricorrente mantiene l'implementazione ispezionabile introducendo al tempo stesso elaborazione sequenziale ordinata, uno stato nascosto non lineare e pesi ricorrenti. Prepara il progetto per versioni successive basate su architetture ricorrenti più avanzate come reti GRU e LSTM.